

Documentação da Arquitetura SOA — 2F-AGRO / Space Connect

Disciplina: SOA — Service-Oriented Architecture · FIAP 3ES · GS 2026.1 **Repositório:** <https://github.com/GS-SPACE-CONNECT/2f-agro-soa>

Integrantes

Nome	RM
(preencher)	(preencher)
(preencher)	(preencher)
(preencher)	(preencher)

GitHub dos responsáveis: @brnleao · @DevRuanVieira · apoio @jota0802

1. Descrição da solução, problema e objetivos

Problema

No agronegócio, decisões de campo (irrigação, proteção contra geada, manejo de pragas) dependem de dados de **clima/satélite** e de **cadastros oficiais do governo** (EMATER/MAPA/CAR). Esses sistemas são heterogêneos: o app moderno fala **REST/JSON**, enquanto o sistema legado do governo expõe **SOAP/XML**. Integrá-los exige interoperabilidade real.

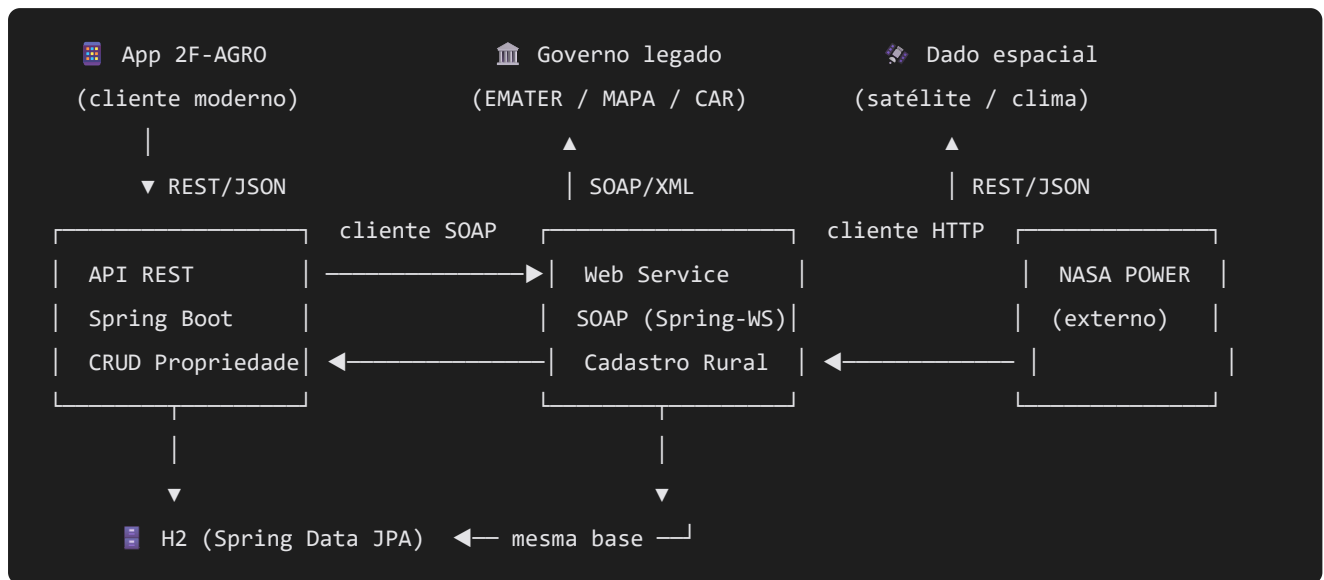
Solução

Uma solução de **serviços distribuídos** em **Java + Spring Boot** que: - expõe uma **API REST** para o CRUD da entidade `Propriedade`; - simula o **sistema legado do governo** com um **Web Service SOAP** (cadastro rural); - **integra** os dois e enriquece a propriedade com **dados espaciais externos** (NASA POWER), gerando alertas agroclimáticos.

Objetivos

- Demonstrar os princípios **SOA**: baixo acoplamento, reutilização, contratos de serviço, interoperabilidade (REST ↔ SOAP) e separação de responsabilidades.
- Aplicar **POO** (abstração, encapsulamento, herança, polimorfismo) no domínio.
- Entregar persistência, validação, tratamento de erros e integração resiliente.

2. Diagrama de Arquitetura SOA



Camadas (pacote `br.com.fiap.agro.soa`): `rest` (controllers) · `soap` (endpoint + cliente) · `service` (regras + integração) · `repository` (JPA) · `domain` (POO) · `dto`.

3. API REST (CRUD de Propriedade)

Base: `http://localhost:8080/api/propriedades`

Método	Caminho	Ação	Status sucesso
GET	<code>/api/propriedades</code>	Lista todas	200
GET	<code>/api/propriedades/{id}</code>	Busca por id	200 (404 se não existe)
POST	<code>/api/propriedades</code>	Cria	201 (+ header <code>Location</code>)
PUT	<code>/api/propriedades/{id}</code>	Atualiza	200 (404 se não existe)
DELETE	<code>/api/propriedades/{id}</code>	Remove	204 (404 se não existe)

Validação (Bean Validation): `produtor/cultura/municipio` obrigatórios, `uf` com 2 letras, `latitude` ∈ [-90,90], `longitude` ∈ [-180,180], `areaHa` > 0. **Erros** centralizados (`@RestControllerAdvice`) com payload padronizado.

Exemplo — POST (201)

Request:

```
{ "produtor": "Joao Silva", "cultura": "Soja", "latitude": -23.5,
  "longitude": -46.6, "areaHa": 120.5, "municipio": "Ribeirao Preto", "uf": "SP" }
```

Response 201 :

```
{ "id": 1, "produtor": "Joao Silva", "cultura": "Soja", "latitude": -23.5,
  "longitude": -46.6, "areaHa": 120.5, "municipio": "Ribeirao Preto", "uf": "SP" }
```

Exemplo — validação (400)

Response 400 :

```
{ "status": 400, "erro": "Bad Request",
  "mensagem": "Falha de validação nos campos enviados",
  "caminho": "/api/propriedades",
  "campos": { "latitude": "latitude deve ser <= 90", "uf": "uf deve ter exatamente 2 letras",
    "produtor": "produtor é obrigatório", "areaHa": "areaHa deve ser positiva" } }
```

4. Web Service SOAP (Cadastro Rural)

Abordagem **contract-first**: o contrato `cadastro-rural.xsd` define os tipos; o Spring-WS publica o **WSDL** dinamicamente.

- **WSDL**: `http://localhost:8080/ws/cadastro-rural.wsdl`
- **Endpoint SOAP**: POST `http://localhost:8080/ws`
- **Namespace**: `http://fiap.com.br/agro/soa/cadastro-rural`

Operação	Entrada	Saída
<code>consultarCadastroRural</code>	<code>cpf</code> e/ou <code>car</code>	<code>encontrado</code> + bloco <code>cadastro</code>
<code>registrarCadastroRural</code>	dados do produtor	<code>protocolo</code> , <code>situacao</code> , <code>mensagem</code>

Exemplo — registrar

Request:

```
<cad:registrarCadastroRuralRequest xmlns:cad="http://fiap.com.br/agro/soa/cadastro-rural">
  <cad:cpf>123.456.789-00</cad:cpf>
  <cad:car>CAR-SP-001</cad:car>
  <cad:nomeProdutor>Joao Silva</cad:nomeProdutor>
  <cad:municipio>Ribeirao Preto</cad:municipio>
  <cad:uf>SP</cad:uf>
  <cad:areaHa>120.5</cad:areaHa>
</cad:registrarCadastroRuralRequest>
```

Response:

```
<ns2:registrarCadastroRuralResponse xmlns:ns2="http://fiap.com.br/agro/soa/cadastro-rural">
  <ns2:protocolo>CAR-1</ns2:protocolo>
  <ns2:situacao>REGISTRADO</ns2:situacao>
  <ns2:mensagem>Cadastro rural registrado com sucesso para Joao Silva</ns2:mensagem>
</ns2:registrarCadastroRuralResponse>
```

5. Integração entre serviços

Endpoint orquestrador: `POST /api/integracao/propriedades`

`POST /api/integracao/propriedades`

- └(1) cria a propriedade REST → H2 (Spring Data JPA)
- └(2) registra no "governo" SOAP → WebServiceTemplate (cliente JAXB) → /ws
- └(3) consulta clima do ponto REST → NASA POWER (RestClient, timeout+fallback)
- └(4) deriva alerta agroclimático .. POO → Alerta.gerarMensagem() (polimorfismo)

- **Serviço externo (espacial):** `NasaPowerServicoClimatico` consome a API *climatology* da NASA POWER via `RestClient` (timeouts 5s/8s) e implementa a abstração `ServicoClimatico`.
- **REST ↔ SOAP interno:** `CadastroRuralClient` (`WebServiceTemplate` + JAXB) chama o próprio Web Service SOAP pela rede.
- **Resiliência:** falha de qualquer serviço externo **não derruba** o fluxo — vira `aviso` no resultado (`climaDisponivel=false` / `protocoloGoverno=null`), e a propriedade é criada.

Exemplo — enriquecida (clima real NASA)

```
{ "propriedade": { "id": 1, "municipio": "Ribeirão Preto", "uf": "SP" },
  "climaDisponivel": true,
  "clima": { "temperaturaMediaC": 23.45, "precipitacaoMm": 3.48, "umidadeRelativa": 68.97, "fonte": "NASA",
  "alerta": null }
```

6. Princípios de POO aplicados

Pilar	Onde
Abstração	<code>interface ServicoClimatico</code> (NASA/CPTEC intercambiáveis)
Encapsulamento	entidades com campos privados + validação nos setters/construtor
Herança	<code>abstract Alerta</code> → <code>AlertaSeca</code> , <code>AlertaGead</code> , <code>AlertaPraga</code>
Polimorfismo	<code>alerta.gerarMensagem()</code> sobrescrito por subtipo

7. Evidências de funcionamento

Os request/response reais (REST JSON + SOAP XML + integração + fallback) estão em `docs/evidencias/EVIDENCIAS.md` , incluindo: - CRUD REST: 200 / 201 / 204 / 400 (validação campo-a-campo) / 404; - SOAP: registrar (protocolo) e consultar (encontrado / não encontrado); - Integração: enriquecimento com clima real da NASA e **fallback resiliente** com serviços fora do ar.

Coleções de teste: **Postman** e **SoapUI**. *(Inserir os prints das execuções nesta seção do PDF.)*

8. Tecnologias utilizadas

- **Java 17, Spring Boot 3.3.5**
 - **Spring Web** (REST), **Spring Web Services / Spring-WS** (SOAP contract-first), **WSDL4J**
 - **Spring Data JPA + H2** (banco em memória)
 - **Bean Validation** (Hibernate Validator)
 - **JAXB** (geração a partir do XSD), **RestClient** (cliente HTTP), **WebServiceTemplate** (cliente SOAP)
 - **Maven**
-

9. Conclusão

O projeto demonstra, de ponta a ponta, os princípios de **SOA**: serviços com contratos bem definidos (REST e WSDL/SOAP), **interoperabilidade** entre um cliente moderno e um sistema legado, **integração** com um serviço externo real (NASA POWER) e **baixo acoplamento** via interfaces e DTOs. A arquitetura em camadas e a aplicação dos pilares de **POO** tornam o código coeso, testável e extensível (ex.: trocar a fonte climática para CPTEC sem alterar o restante). O tratamento resiliente de falhas garante disponibilidade do fluxo principal mesmo quando serviços externos ficam indisponíveis.